

DOKUMENTACJA I KOD ŹRÓDŁOWY APLIKACJI MOVO

Niniejszy dokument zawiera kompletną konfigurację, instrukcję wdrożenia oraz kluczowe pliki źródłowe aplikacji MOVO. System został zaprojektowany w oparciu o framework Next.js oraz backend Supabase.

1. ZMIENNE ŚRODOWISKOWE

Poniższe zmienne należy umieścić w pliku `.env.local` w katalogu głównym projektu. Są one niezbędne do poprawnego nawiązania połączenia z bazą danych i usługami uwierzytelniania.

- `NEXT_PUBLIC_SUPABASE_URL` = `& Person` Twój adres Supabase
- `NEXT_PUBLIC_SUPABASE_ANON_KEY` = `sb_publishable_t0tgM1h7rYBdWAD23H7DdQ_JG9Nb0_G`

2. INSTRUKCJA WDROŻENIA W VERCEL

Proces wdrożenia aplikacji został zoptymalizowany pod platformę Vercel i wymaga wykonania trzech głównych kroków.

Krok 1: Wgrywanie plików na GitHub

- Przejdź do swojego repozytorium GitHub o nazwie **MOVO**.
- Użyj funkcji **Add file** i wybierz **Upload files**.
- Prześlij strukturę katalogów projektu lub wklej zawartość plików opisaną w dalszej części dokumentu.
- Zatwierdź zmiany przyciskiem **Commit changes**.

Krok 2: Konfiguracja projektu w Vercel

- Zaloguj się do panelu Vercel za pomocą konta GitHub.
- Wybierz **Add New...** a następnie **Project**.
- Z listy repozytoriów wybierz **MOVO** i kliknij **Import**.

Krok 3: Konfiguracja środowiska i Deployment

- W sekcji **Environment Variables** dodaj klucze zdefiniowane w punkcie pierwszym:
 - `NEXT_PUBLIC_SUPABASE_URL`
 - `NEXT_PUBLIC_SUPABASE_ANON_KEY`
- Zainicjuj proces budowania przyciskiem **Deploy**.

3. GŁÓWNE PLIKI APLIKACJI

A. Konfiguracja Supabase (src/lib/supabase/client.ts)

Ten plik inicjuje klienta po stronie przeglądarki, umożliwiając interakcję z bazą danych bezpośrednio z komponentów React.

```
import { createBrowserClient } from '@supabase/ssr';
```

```
export function createClient() {  
  
  return createBrowserClient(  
  
    process.env.NEXT_PUBLIC_SUPABASE_URL!,  
  
    process.env.NEXT_PUBLIC_SUPABASE_ANON_KEY!  
  
  );  
  
}
```

B. Komponent Mówuś (src/components/MowusWidget.tsx)

Widget asystenta, który dostarcza użytkownikowi dynamiczne komunikaty oraz informacje o postępie w programie lojalnościowym.

```
'use client';
```

```
import React, { useState } from 'react';  
  
interface MowusProps {  
  
  message?: string;
```

```

pointsToNextLevel?: number;

}

export default function MowusWidget({ message, pointsToNextLevel }:
MowusProps) {

  const [isOpen, setIsOpen] = useState(true);

  const defaultMessage = pointsToNextLevel

    ? `Jeszcze tylko ${pointsToNextLevel} pkt i zdobywasz kolejny poziom w
MOVO! 🚀`

    : 'Cześć! Jestem Mówuś. Czego dzisiaj potrzebujesz? 🍕📦';

  return (

    <div className="fixed bottom-5 right-5 z-50 flex items-end gap-3">

      {isOpen && (

        <div className="bg-white border-2 border-yellow-400 p-4 rounded-2xl
shadow-xl max-w-xs animate-bounce-once relative">

          <button

            onClick={() => setIsOpen(false)}

            className="absolute top-1 right-2 text-gray-400 hover:text-black
font-bold text-sm"

            >

              ✕

            </button>

```

```

        <p className="text-sm font-semibold text-gray-800">{message ||
defaultMessage}</p>

|---|

    )}

    <div

        onClick={() => setIsOpen(!isOpen)}

        className="w-16 h-16 bg-yellow-400 border-4 border-black rounded-full
flex items-center justify-center cursor-pointer shadow-2xl hover:scale-105
transition-transform"

        >

        <span className="text-3xl">🐱⚡ </span>

    </div>

</div>

);

}

```

C. Formularz Zamówień

(src/app/customer/orders/new/page.tsx)

Główny interfejs składania zamówień, obsługujący pobieranie listy miast oraz walidację danych wejściowych od klienta.'use client';

```
import React, { useState, useEffect } from 'react';
```

```
import { useRouter } from 'next/navigation';
```

```
import { createClient } from '@lib/supabase/client';
```

```
export default function NewOrderPage() {

  const router = useRouter();

  const supabase = createClient();


  const [cities, setCities] = useState<any[]>([]);

  const [selectedCity, setSelectedCity] = useState<string>('');

  const [merchantName, setMerchantName] = useState('');

  const [itemsDescription, setItemsDescription] = useState('');

  const [itemsValue, setItemsValue] = useState<number>(0);

  const [pickupAddress, setPickupAddress] = useState('');

  const [deliveryAddress, setDeliveryAddress] = useState('');

  const [deliveryFee, setDeliveryFee] = useState<number>(17.00);

  const [loading, setLoading] = useState(false);

  useEffect(() => {

    async function fetchCities() {

      const { data } = await supabase.from('cities').select('*');

      if (data && data.length > 0) {

        setCities(data);

        setSelectedCity(data[0].id);

      }

    }

  });

}
```

```
}
```

```
fetchCities();
```

```
}, []);
```

```
const handleSubmit = async (e: React.FormEvent) => {
```

```
  e.preventDefault();
```

```
  setLoading(true);
```

```
  const { data: { user } } = await supabase.auth.getUser();
```

```
  if (!user) return;
```

```
  const totalAmount = Number(itemsValue) + Number(deliveryFee);
```

```
  const { data, error } = await supabase.from('orders').insert({
```

```
    customer_id: user.id,
```

```
    city_id: selectedCity,
```

```
    merchant_custom_name: merchantName,
```

```
    items_description: itemsDescription,
```

```
    items_value: itemsValue,
```

```
    delivery_fee: deliveryFee,
```

```
    total_amount: totalAmount,
```

```
    pickup_address: pickupAddress,
```

```
    delivery_address: deliveryAddress,
```

```
    status: 'AWAITING_PAYMENT',
```

```
    payment_method: 'BLIK_PHONE'

  }).select().single();

  if (error) {

    alert('Błąd tworzenia zamówienia: ' + error.message);

    setLoading(false);

  } else {

    router.push(`/customer/orders/${data.id}`);

  }

};

return (

  <div className="max-w-2xl mx-auto p-6 bg-white rounded-xl shadow-md my-8">

    <h1 className="text-2xl font-bold mb-6 text-gray-900">Złóż nowe  
zamówienie w MOVO 📦</h1>

    <form onSubmit={handleSubmit} className="space-y-4">

      <div>

        <label className="block text-sm font-medium text-gray-700">Wybierz  
Miasto</label>

        <select

          value={selectedCity}

          onChange={(e) => setSelectedCity(e.target.value)}


```

```
      className="w-full mt-1 p-2 border border-gray-300 rounded-md"

    >

      {cities.map((city) => (

        <option key={city.id} value={city.id}>{city.name}</option>

      ))}

    </select>

  </div>

  <div>

    <label className="block text-sm font-medium text-gray-700">Skąd
    odbieramy? (Sklep, Punkt, Paczkomat)</label>

    <input

      type="text"

      required

      placeholder="np. McDonald's, Apteka, Biedronka"

      value={merchantName}

      onChange={(e) => setMerchantName(e.target.value)}

      className="w-full mt-1 p-2 border border-gray-300 rounded-md"

    />

  </div>

  <div>
```

```
<label className="block text-sm font-medium text-gray-700">Dokładny  
adres odbioru</label>
```

```
<input  
  
  type="text"  
  
  required  
  
  placeholder="np. ul. Rynek 12, Namysłów"  
  
  value={pickupAddress}  
  
  onChange={(e) => setPickupAddress(e.target.value)}  
  
  className="w-full mt-1 p-2 border border-gray-300 rounded-md"  
  
>
```

```
</div>
```

```
<div>
```

```
<label className="block text-sm font-medium text-gray-700">Co mamy  
kupić / odebrać?</label>
```

```
<textarea  
  
  required  
  
  rows={3}  
  
  placeholder="Wpisz listę produktów lub numer odbioru paczki"  
  
  value={itemsDescription}  
  
  onChange={(e) => setItemsDescription(e.target.value)}  
  
  className="w-full mt-1 p-2 border border-gray-300 rounded-md"
```

```
/>
```

```
</div>
```

```
<div>
```

```
  <label className="block text-sm font-medium text-gray-700">Szacowana kwota zakupów (zł)</label>
```

```
  <input
```

```
    type="number"
```

```
    step="0.01"
```

```
    required
```

```
    value={itemsValue}
```

```
    onChange={(e) => setItemsValue(Number(e.target.value))}
```

```
    className="w-full mt-1 p-2 border border-gray-300 rounded-md"
```

```
  />
```

```
</div>
```

```
<div>
```

```
  <label className="block text-sm font-medium text-gray-700">Adres dostawy do Ciebie</label>
```

```
  <input
```

```
    type="text"
```

```
    required
```

```
    placeholder="np. ul. Słoneczna 4/12"
```

```
      value={deliveryAddress}

      onChange={(e) ⇒ setDeliveryAddress(e.target.value)}

      className="w-full mt-1 p-2 border border-gray-300 rounded-md"

    />

  </div>

  <button

    type="submit"

    disabled={loading}

    className="w-full bg-yellow-400 text-black font-bold py-3 rounded-lg
    hover:bg-yellow-500 transition-colors"

    >

      {loading ? 'Przetwarzanie...' : 'Złóż zamówienie i przejdź do płatności'}

    </button>

  </form>

</div>

);

}
```